Lab 04

Object Oriented Programming

BS DS/CY Fall 2025 Morning/Afternoon

Instructions:

- Indent your code properly.
- Use meaningful variable names and follow standard naming conventions.
- Use clear and meaningful prompts/labels for all program input and output.
- Make sure that there are NO dangling pointers or memory leaks in your program.
- Do not discuss your problems with fellow students. If you face any difficulty in understanding, consult your TAs.

Task 01

A *ragged array* is an array which contains a varying number of elements in each row. The *Pascal triangle* can be used to compute the coefficients of the terms in the expansion of $(a+b)^n$. In a Pascal triangle, each element is the sum of the element directly above it (if any) and the element to the left of the element directly above it (if any). A Pascal triangle of size 7 is shown below:

1						
1	1					
1	2	1				
1	3	3	1			
1	4	6	4	1		
1	5	10	10	5	1	
1	6	15	20	15	6	1

You are required to implement the following functions.

Task 1.1

int createPascalTriangle (int n);**

This function will take an integer (**n**) as argument and create a Pascal triangle consisting of **n** rows. It will dynamically allocate the two-dimensional *ragged array*, fill up its elements, and return a pointer to this filled array (Pascal triangle).

Task 1.2

void displayPascalTriangle (int pt, int n);**

This function will take a pointer **pt** which is pointing to a Pascal triangle consisting of **n** rows. It will display the Pascal triangle on screen.

Task 1.3

void deallocatePascalTriangle (int pt, int n);**

This function will take a pointer **pt** which is pointing to a Pascal triangle consisting of **n** rows. This function will deallocate the two-dimensional array containing the Pascal triangle.

Task 1.4

Write a main function which asks the user to specify the value of **n**. After that the main should call the above functions to create a Pascal triangle of size **n**, display it on screen, and, finally, deallocate all the dynamically allocated memory.

Task 02

Matrix Application

In this problem, our goal is to design a library, which will support basic operations of Matrices.
You have following structure for matrix

```
struct Matrix{

        int rows;
        int cols;
        int **data
};
```

You are required to design the following operations.

1. void createMatrix (Matirx & m, const int row=1, const int Col=1);

2. int& at(Matrix m, const int r , const int c);

//For setting or getting some value at a particular location of matrix

3. void printMatrix(const Matrix m)

4. int isIdentity (const Matrix m)
if a_{ij} = 0 for i != j and a_{ij} = 1 for all i = j .

5. bool isRectangular (const Matrix m)
In which number of rows are not equal to number of columns.

6. bool isDiagonal (const Matrix m)
I f a_{ij} = 0 for all i != j and at least one a_{ij} != 0 for i = j;

7. bool isNullMatrix (const Matrix m)
A matrix whose each element is zero.

8. bool isLowerTriangular (const Matrix m)

9. bool isUpperTriangular (const Matrix m)

10. bool isTriangular (const Matrix m)

11. Matrix getMatrixCopy (const Matrix m)

12. bool isEqual(const Matrix m1 , const Matrix m2)

13. void freeMatrix (Matrix & m);
Free the dynamically allocated memory.

14. Matrix Transpose (const Matrix m);

15. Matrix minus (const Matrix m1, const Matrix m2);

16. void reSize (Matrix & m, const int newrow, const int newcol);

17. bool isSymmetric (const Matrix m)
If A^t = A

18. bool isSkewSymmetric (const Matrix m)
If A^t = -A

19. Matrix add (const Matrix m1, const Matrix m2);

20. Matrix multiply(const Matrix m1, const Matrix m2);

Also write down the main function to demonstrate your implementation.